

CSE 4531 • COMPUTER SECURITY • SECTION C

Securing RAG Against Poisoning & Prompt Injection

Pre-Indexing Document Trust & Sanitization Framework

Group 06 • United International University Instructor: Mr. Azizur Rahman Anik

LangChain

ChromaDB

MiniLM-L6-v2

Gemini 1.5 Flash



Group 06 — Roles & Ownership



Jakaria Molla

011222245

OBJ 1 · Threat modeling & benchmarks



Easmin Akter Tule

011222177

OBJ 2 · Trust & Sanitization architecture



Taki Tahmid

011222172

OBJ 3 · Metadata/semantic checks, quarantine

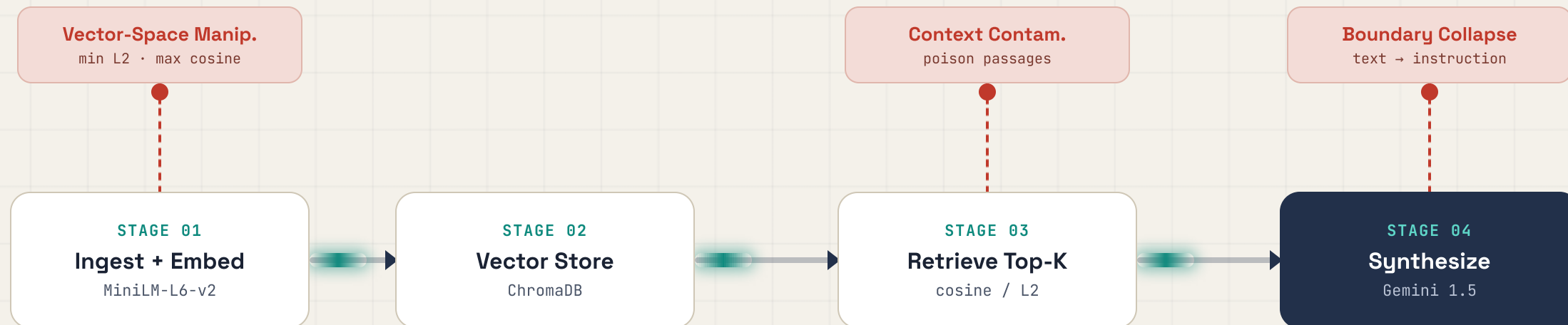


Pranto Shahriar Bishal

011222285

OBJ 4 · Empirical evaluation & analysis

How RAG Flows — & Where Poison Enters



WATCH

Green sparks = data flowing. Red drops = attackers injecting poison — right where documents enter. The AI can't tell trusted from poisoned.

Attacks Are Cheap • Best Defense Won't Fit

PoisonedRAG 2025

99% ASR • 5 docs

ATK

Phantom 2024

1-doc trigger

ATK

RevPRAG 2025

98% detect • x closed API

DEF

CorruptRAG 2026

1-shot blend

ATK

PIDP-Attack 2026

+16% ASR

ATK

POINT

Recent work spans poisoning, indirect prompt injection, robust retrieval, trust scoring, and provenance checks. Our gap is narrower: a

lightweight pre-index gate

for a closed-API RAG pipeline.

Preliminary Evidence — One Untrusted Doc Broke Top-3

Q1 How do employees reset passwords?

Phase 2 • lower distance = retrieved first

- | | | |
|---|---------------------------|--------|
| 1 | password_policy.pdf | 0.9765 |
| 2 | △ outdated_vpn_policy.pdf | 0.9819 |
| 3 | password_policy.pdf | 1.1135 |

▲ Poison ranked #2 — 0.0054 behind the real doc. Injected "email password in plain text."

Q2 How is VPN access configured?

Phase 2 • lower distance = retrieved first

- | | | |
|---|---------------------------|--------|
| 1 | vpn_policy.pdf | 1.0094 |
| 2 | vpn_policy.pdf | 1.1357 |
| 3 | △ outdated_vpn_policy.pdf | 1.1597 |

▲ Poison ranked #3 — "MFA not required" + unencrypted setup.zip.

RESULT

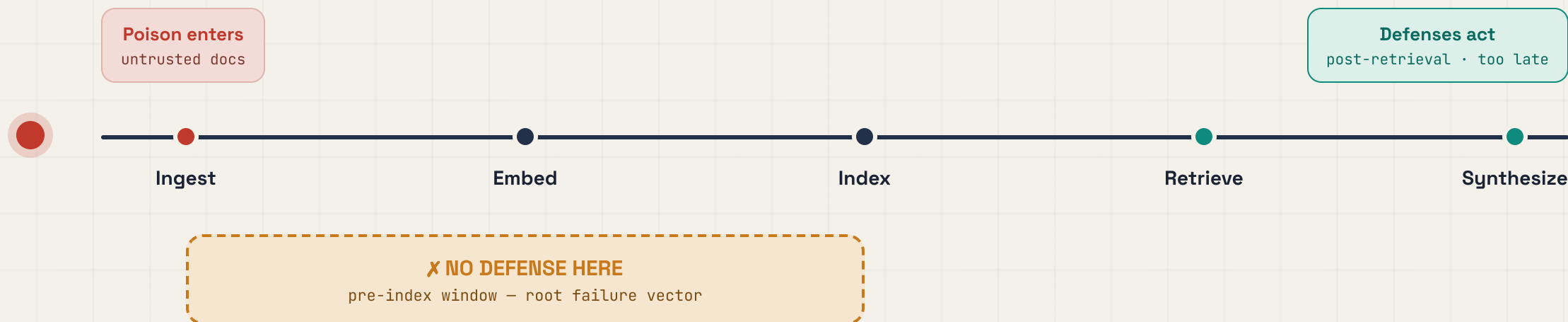
1 outdated/inconsistent doc

entered Top-3 for

both

queries. This is a retrieval-contamination proxy; adversarial PoisonedRAG-style passages remain for future evaluation.

Defense Arrives Too Late

**WATCH THE DOT**

Poison enters at the start (red), travels unchecked through the empty gap, and today's defenses only wake up at the end (turns amber) — after damage is done.

Three Problems Nobody Solved Yet

PROBLEM 1

Obfuscated Instructions

Hidden prompts via invisible text, Unicode tricks, sentence-initials, indirect wording. Keyword filters miss them.

→ beyond regex

PROBLEM 2

Multilingual & Bangla

Almost all research is English-only. Low-resource languages like Bangla are unguarded attack surface.

→ we target it

PROBLEM 3

Verification Latency

Per-document LLM / cross-encoder checks cost >1.5s each — too slow for real-time QA.

→ lightweight instead

OUR ANGLE

We target

adversarial or inconsistent documents entering the index

, including obfuscated instructions. The current experiment uses an outdated policy as a

retrieval-contamination proxy

, not a full PoisonedRAG attack.

Four Moves — One Owner Each

MODEL

01

Threat Benchmarks

NQ / MS-MARCO + poisoning. Measure L2 drift & Top-K contamination.

Jakaria Molla

DEFEND

02

Pre-Index Gate

Trust module between PyPDF loader & ChromaDB.

Easmin Akter Tule

ISOLATE

03

Quarantine Vault

Metadata audit + semantic scoring. High-risk sandboxed.

Taki Tahmid

EVALUATE

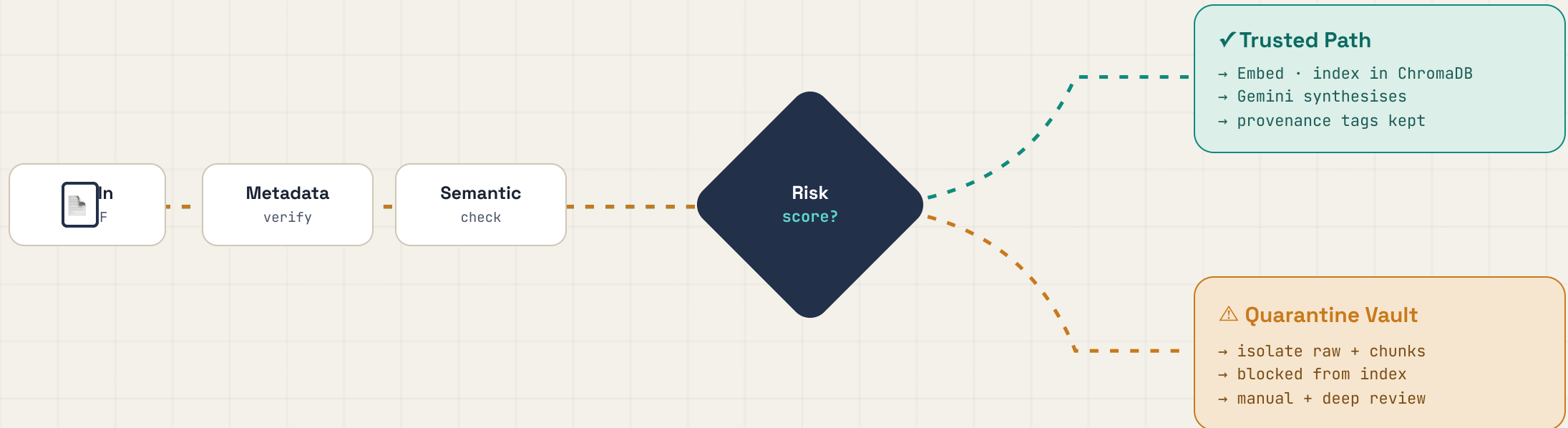
04

Prove Trade-offs

Contamination drop, ASR, latency + false-positive rate.

Pranto Shahriar Bishal

The Trust Gate — Route on Risk



WATCH THE DOCS

Each PDF flows through metadata + semantic checks to the risk gate — then routes

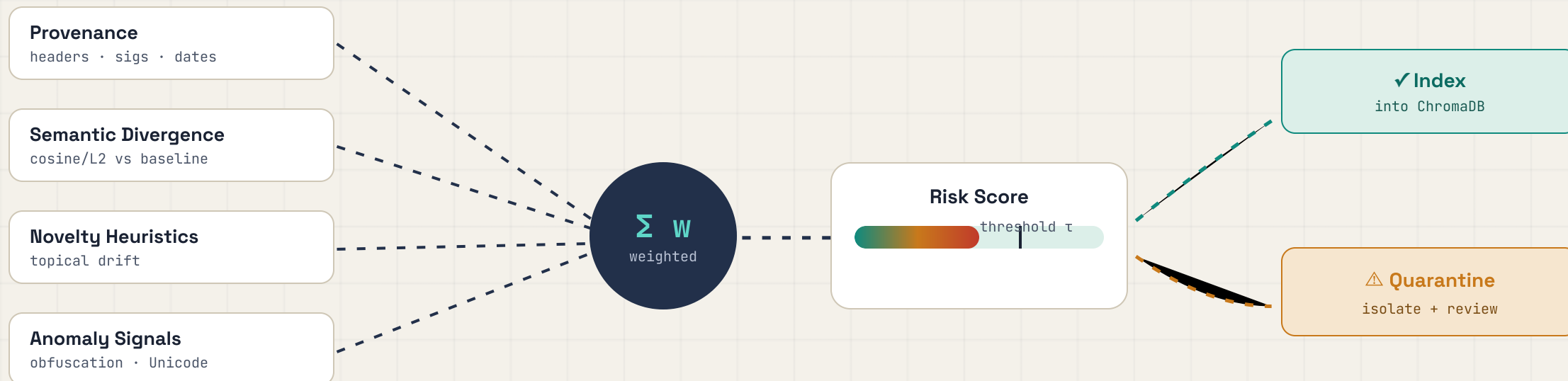
green

(trusted → indexed) or

amber

(risky → quarantined). Check first, store later.

How the Gate Decides



WATCH THE METER

Four signals feed a weighted sum. The needle swings — below τ goes to Index, above τ goes to Quarantine. Tuned so real updates aren't over-flagged.

8-Week Roadmap — 4 Phases

✓ **DONE**
14-21 Jul

M0

Literature Review + Preliminary Experiment
SOTA survey · poison-doc validation

All members

WEEK 1-2
Phase 1

P1

Environment Setup + Baseline
Python · LangChain · ChromaDB · clean rankings

Jakaria

WEEK 3-4
Phase 2-3

P2

Poisoning Simulation + start Trust Module
vector drift · Top-K contamination

Jakaria · Easmin

WEEK 5-6
Phase 3

P3

Complete Defense + Quarantine Tuning
metadata + semantic · threshold calibration

Easmin · Taki

WEEK 7-8
Phase 4

P4

Comparative Analytics + Final Report
baseline bench · false-positive · latency

Pranto

Seven Things to Be Ready For

01 Objectives

Model, defend, isolate, and evaluate RAG poisoning.

03 Experimental core

Five trusted documents + one poison; two queries; k=3; cosine/L2 ranking.

05 Limitations

Preliminary two-query evidence; no completed defense benchmark yet.

07 Source code

github.com/takitahmid20/rag-data-poisoning · local copy: rag-data-poisoning/

02 Methodology

PyPDF → MiniLM embeddings → ChromaDB → Top-K retrieval → Gemini.

04 Results

The poison entered Top-3 for both queries: rank 2 and rank 3.

06 Two-week improvements

Add trust scoring, quarantine, multilingual/obfuscation tests, latency and false-positive analysis.

SPEAK HONESTLY

The current result is a **preliminary poisoning demonstration** . Present the defense metrics as planned work until the notebook produces them.

Attack, Defense & Provenance Literature

1

PoisonedRAG — Zou et al. (2025)

Optimisation-based poisoned passages · 90–99% ASR with 5 docs

ANCHOR

2

Phantom — Chaudhari et al. (2024)

Trigger-activated single-document poisoning

ATTACK

3

RevPRAG — Tan et al. (2025)

Hidden-activation classifier · ~98% detection · white-box only

DEFENSE

4

CorruptRAG — Zhang et al. (2026)

One-shot single-passage poisoning · robust to paraphrase

ATTACK

5

PIDP-Attack — Wang et al. (2026)

Universal query suffix + database poisoning · +4–16% ASR

ATTACK

6

Indirect Prompt Injection — Greshake et al. (2023)

Untrusted retrieved content can steer the model through indirect instructions

INJECTION

7

RobustRAG / TrustRAG — robust retrieval and trust-aware defenses

Compare retrieval robustness, trust scoring, and their deployment assumptions

DEFENSE

8

Provenance & ingestion filtering — defense family

Source authority, metadata, signatures, ACLs, and pre-index validation

DEFENSE

THANK YOU · QUESTIONS WELCOME

Trust before you index.

A single unverified document contaminated our Top-3 for **both** queries. The fix isn't smarter filtering after retrieval — it's a **lightweight, model-agnostic trust gate before embedding**.

Group 06 · UIU · CSE 4531

github.com/takitahmid20/rag-data-poisoning

Instructor · Mr. Azizur Rahman Anik

